

Process Management

2026 Semester 1 SOFTENG 370: Operating Systems

Talia Xu

Lecture 6

2.0.0

Based on original slides by Professor Jon Eyolfson.

This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/)

The Operating System Creates Processes

The operating system has to:

- Maintain process control blocks, including state
- Create new processes
- Load a program, and re-initialize a process with context

On Unix, the Kernel Launches A Single User Process

After the kernel initializes, it creates a single process from a program

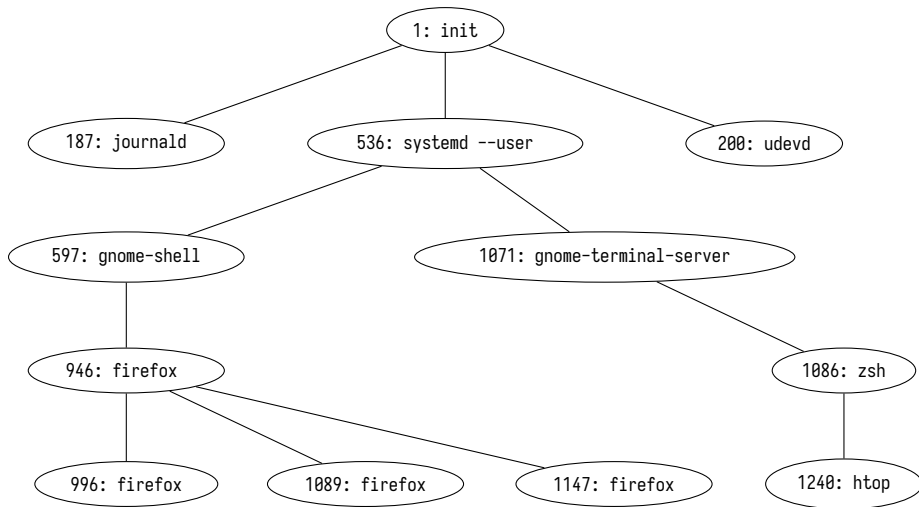
This process is called `init`, and it looks for it in `/sbin/init`

- Responsible for executing every other process on the machine

- Must always be active, if it exits the kernel thinks you're shutting down

For Linux, `init` will probably be `systemd` but there's other options

A Typical Process Tree



Processes Are Assigned a Process ID (pid) On Creation

The process ID is just a number, and is unique for every **active** process

On most Linux systems the maximum pid 32768, and 0 is reserved (invalid)

Eventually the kernel will recycle a pid, after the process dies, for a new process

Remember: each process has its own *address space* (independent view of memory)

Maintaining the Parent/Child Relationship

Previously, we made sure that our parent exited last (by using sleep)

What happens if the parent process exits first, and no longer exists?

The Parent Process is Responsible for Its Child

The operating system sets the exit status when a process terminates (the process terminates by calling `exit`)

It can't remove its PCB yet

The minimum acknowledgment the parent has to do is read the child's exit status

There's two situations:

1. The child exits first (zombie process)
2. The parent exits first (orphan process)

A Zombie Process Waits for Its Parent to Read Its Exit Status

The process is terminated, but it hasn't been acknowledged

A process may have an error in it, where it never reads the child's exit status

The operating system can interrupt the parent process to acknowledge the child

- It is just a suggestion and the parent is free to ignore it

- This is a basic form of IPC called a signal

The operating system has to keep a zombie process until it's acknowledged

- If the parent ignores it, the zombie process needs to wait to be re-parented

An Orphan Process Needs a New Parent

The child process lost its parent process

The child still needs a process to acknowledge its exit

The operating system re-parents the child process to init

The init process is now responsible to acknowledge the child

orphan-example.c The Parent Exits Before the Child

```
int main(int argc, char *argv[]) {
    pid_t pid = fork();
    if (pid == 0) {
        printf("Child parent pid: %d\n", getppid());
        sleep(2);
        printf("Child parent pid (after sleep): %d\n", getppid());
    }
    else {
        sleep(1);
    }
    return 0;
}
```

wait delays execution

wait uses the following API:

- status: Address to store the wait status of the process
- Returns the process ID of child process
 - 1: on failure
 - >0: the child with a change

The wait status contains a bunch of information, including the exit code

Use `man wait` to find all the macros to query wait status

You can use `waitpid` to wait on a specific child process

wait Example

```
int main() {  
    int pid = getpid();  
    fork();  
    if (getpid() == pid){  
        wait(NULL);  
        printf("Hello A\n");  
    }  
    else{  
        printf("Hello B\n");  
    }  
    return 0;  
}
```

zombie-example.c The Parent Monitors the Child To Check Its State

```
pid_t pid = fork();

if (pid == 0)
    sleep(2);

else {
    int ret;
    sleep(1);
    printf("Child process state: ");
    ret = print_state(pid);
    sleep(2);
    printf("Child process state: ");
    ret = print_state(pid);
}
```

You're Responsible for Managing Processes

The operating system maintains a strict parent/child relationship

You should be able to identify (and prevent) the following:

- Zombie processes
- Orphan processes

Review Question

A process P1 runs on a Linux-like OS with a single core system.

When P1 is executing, a disk interrupt occurs, causing P1 to go to kernel mode to service that interrupt.

The interrupt delivers all the disk blocks that unblock a process P2 (which blocked earlier on the disk read).

The interrupt service routine has completed execution fully, and the OS is just about to return back to the user mode of P1.

At this point in time, what are the states (ready/running/blocked) of processes P1 and P2?

Review Question

A process P invokes the default wait system call. Which of the following scenarios block P (i.e., P does not return immediately)?

- (a) P has no children.
- (b) P has one child that is still running.
- (c) P has one child that is a zombie.
- (d) P has two children, one is running and one is a zombie.

Review Question

What sequence of system calls is executed when you run "sleep 100" and press Enter?

- (a) wait-exec-fork
- (b) exec-wait-fork
- (c) fork-exec-wait
- (d) wait-fork-exec

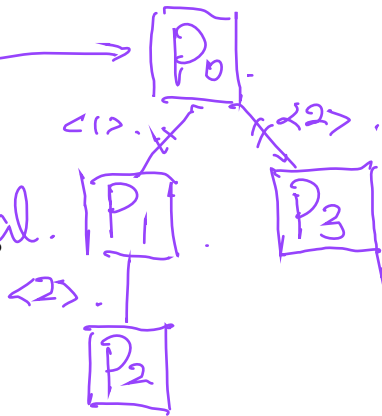
Review Question

A process P runs the following code:

```
<0> int ret1 = fork();  
<1> int ret2 = fork();  
<2>
```

← 2x
← 4x in total.

How many direct children does P have?



Review Question

What is the behavior of this program?

```
int main() {  
    char *command = "/bin/ls";  
    char *args[] = {"ls", NULL};  
    int rc = fork();  
    if (rc == 0) {  
        close(STDOUT_FILENO);  
        int fd = open("foo.txt");  
        execve(command, args, NULL);  
        exit(0);  
    } else {  
        wait(NULL);  
    }  
    return 0;  
}
```

child.

create child process.

(stdin, stdout, stderr).

→ Pipe -

will not executed.

parent

open foo.txt, run ls
write output of ls
into file -

(0, 1, 2).

closed. fd becomes
available.

Review Question

6 times.

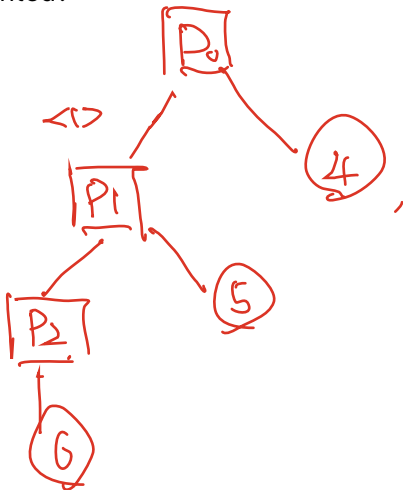
How many times is "Hello world!" printed?

```
#include <stdio.h>
#include <unistd.h>

int main() {
    int value = 2;
    if (fork() == 0) <1>
        fork(); <2>
        fork(); <3>
        printf("Hello, world!\n");
        return 0;
}
```

only child - →

↑



Review Question

Will it produce the same output each time it is run? If so, what is the output?

```
int main() {  
    int i = 4;  
    while (i != 0) {  
        int pid = fork();  
        if (pid == 0) {  
            i--;  
        } else {  
            printf("%d\n", i);  
            exit(0);  
        }  
    }  
    return 0;  
}
```

Review Question

Will it produce the same output each time it is run? If so, what is the output?

```
int main() {  
    int i = 4;  
    while (i != 0) {  
        int pid = fork();  
        if (pid == 0) {  
            i--;  
        } else {  
            waitpid(pid, NULL, 0);  
            printf("%d\n", i);  
            exit(0);  
        }  
    }  
    return 0;  
}
```

1
2
3
4

$P_0(\bar{i}=4) \rightarrow \text{wait}(P_{0 \rightarrow 1})$

$P_{0 \rightarrow 1}(\bar{i}=4) \rightarrow P_{0 \rightarrow 1}(\bar{i}=3) \rightarrow \text{wait}(P_2)$

$P_{1 \rightarrow 2}(\bar{i}=3) \rightarrow P_{1 \rightarrow 2}(\bar{i}=2)$

$\dots P_{2 \rightarrow 3}(\bar{i}=1)$

$P_{3 \rightarrow 4}(\bar{i}=0) \rightarrow \text{return}$

unblock.

Review Question

$x=3, x=3, \dots$

inf. loop.

What is the output?

```
int x = 3;
void main() {
    while (x > 0) {
        int ret = fork();
        char *args[] = {"a.out", NULL};
        if (ret == 0) {
            printf("x = %d\n", x);
            x--;
            execve("./a.out", args, NULL);
        } else {
            wait(NULL);
        }
    }
}
```

a.out

← execution point.

does not propagate
to the next iteration
b/c process is replaced.

Review Question

What is the output?

```
int x = 3;
void main()
{
    while(x > 0) {
        int ret = fork();
        if(ret == 0) {
            printf("x = %d\n", x);
        } else
            wait(NULL);
        x--;
    }
}
```